Employee sign-in with HRMS: design and set-up

Applies to: POGovApi and the Admin portal (this repository).

In one line: employees sign in with their **Employee ID** and **HRMS password**. **Only employees the administrator registered** can sign in. The portal calls only POGovApi; POGovApi calls HRMS for the password **only for registered, active employees**, and a sign-in never creates or changes a user.

---

1. How QPS signs people in (analysed from `QPS.slnLaunch.zip`)

Step	QPS code	What happens
Sign-in page	`Admin/Components/Pages/Account/Login.razor` (`/Account/Login`)	Employee / Vendor tabs, CAPTCHA, then `AccountService.ValidateCredentialsAsync`
User lookup	`Admin/Auth/AccountService.cs`, `AuthUserStoreService.FindByLoginAsync` → API `FindUserByLogin` → `[auth].[PRO_LOGIN_LOOKUP]`	Refuses unknown users before the password check (good), local lockout
Password	`Admin/Auth/EmployeeCredentialVerifier.cs` (`EmployeeApiVerifier`)	**The portal** calls HRMS `HRP_GET_EMP_LOGIN_API` (`{loginID, loginPassword}` → `{responseCode, responseMessage, employeeList[]}`; 104 = HRMS lock)
After HRMS success	same file → `GET_EMP_COP_LOGIN_API` → API `CopLogin` → `PRO_USER_LOGIN`	Sends the HRMS details of the employee to the database on every sign-in
User creation	`UserMasterDialog.razor` → `UserDirectoryService.FindUserByEmpCodeAsync`	Admin types the Employee ID; CO Portal first, then HRMS `HRP_GET_EMP_DETAIL_CODE` from **the portal**

Gaps found

1. **The old sign-in page is still live.** `Admin/Components/Account/Pages/QPSLogin.razor` has `@page "/Account/LoginOld"` and `[AllowAnonymous]`. It calls HRMS for **any** employee code, with no check that the person is a QPS user. It then calls `GET_EMP_COP_LOGIN_API` / `PRO_USER_LOGIN` with that person's HRMS details and signs them in with whatever comes back. This is the loophole: any employee with an HRMS password can try QPS, and the database procedure decides what happens. That procedure's SQL is not in the zip; it receives the full employee record plus `Role_Id`, so check whether it inserts users.
2. **The portal talks to HRMS directly.** The HRMS URLs and credentials are in the portal's API list (`AllApi.ApiDetails`), so every portal server holds the HRMS secret.
3. **Every sign-in writes HRMS data** through `PRO_USER_LOGIN`, even in the new flow. A sign-in should only read.

2. How PO Governance does it

```
 Browser ──► Portal /Account/SignIn ──► POGovApi AuthLogin {loginId, password}
                                                  │
                                                  ├─ 1. Employee ID registered here, active, not locked?
                                                  │       no  → "Employee ID or password is incorrect"
                                                  │             (HRMS is NOT called; audit "not a
registered employee")
                                                  ├─ 2. HRMS HRP_GET_EMP_LOGIN_API (server to server)
                                                  │       wrong password → local failure count (5 →
locked 15 min)
                                                  │       code 104       → "HRMS account locked" (unlock
in MyVishal)
                                                  │       no answer      → "temporarily unavailable"
(not counted)
                                                  ├─ 3. HRMS answered about the same code, employee
active, not resigned?
                                                  └─ 4. Session token → portal cookie. Nothing is
created or changed.
```

Rule	Where
Only registered employees: the user is looked up by Employee ID **before** HRMS is called	`AccountsService.EmployeeLoginAsync`
A sign-in never creates or updates a user (only failure count / last sign-in)	same
An email in the Employee ID field is refused. Exception: the built-in admin, while `AllowAdminLocalLogin` is on	`AccountsService.LoginCoreAsync`
HRMS must answer for the same Employee ID, active and not resigned	same
Every failure takes at least 1 s (`LoginFailureMinMs`), so response times do not reveal registered IDs	`AccountsService.LoginAsync`
Lockout is checked locally first, so a locked account never reaches HRMS	same
The portal never calls HRMS and never sees its URL or credentials	API_MASTER `Api_Scope = SERVER` (§3)
Registration: HRMS details are fetched **by POGovApi**; the portal's copy is never trusted	`UserAdd` → `AccountsService.AddEmployeeAsync`
Unknown, resigned, inactive or already-registered Employee IDs cannot be added	same
Employees have no local password (reset / change point to MyVishal)	`ResetPasswordAsync`, `ChangePasswordAsync`
Every attempt is in the audit log (`login`, `login-fail` with the reason, `login-error`, `user-add`, `user-link-hrms`)	Audit log page

Users. A login is keyed by the employee's HRMS email (SCR scopes and approvals use emails) and carries `empCode` plus the HRMS details (`hr`: department, designation, location, mobile, manager, grade, status, fetched at / by). Logins without an Employee ID, for example those created from the SCR file, show ⚠ **Employee ID needed** and cannot sign in until the admin links one (**Edit → Fetch from HRMS → Link**, or **+ Add user** with the same employee).

New API actions (admin only): `UserHrmsLookup` (`PO_USER_HRMS_LOOKUP`) and `UserLinkEmployee` (`PO_USER_LINK_EMPLOYEE`). `UserAdd` and `UsersBulk` now take `empCode`. `AuthLogin` takes `loginId` (`email` is still read). `Health` reports the sign-in mode, so the sign-in page asks for the Employee ID or the email.

3. Set-up

3.1 HRMS endpoints (once per database)

`database/sqlserver/04_api_master.sql` adds:

- the column `API_MASTER.Api_Scope` (`PORTAL` / `SERVER`);
- two **server-only** rows, `HRP_GET_EMP_LOGIN_API` and `HRP_GET_EMP_DETAIL_CODE`, under `Base_Url_Key = HRMS_API`;
- inactive placeholder base URLs for Development, Staging and Production.

`PRO_GET_ALL_API` never returns `SERVER` rows to the portal (`ALL` / `BY_TASK` / `GetSpecificApi`), and the `*` token of `API_AUTHORIZATION` is never applied to them. Use the HRMS URLs of the QPS API_MASTER rows with the same task names:

```
UPDATE dbo.API_BASE_URL SET Base_Url = N'https://<hrms host>/', Active_Status = 1, Updated_At =
SYSDATETIME()
WHERE Base_Url_Key = N'HRMS_API' AND Environment = N'Production';          -- and Staging /
Development
UPDATE dbo.API_MASTER SET Api_Url = N'<path of the login API>'          WHERE Task_Name =
N'HRP_GET_EMP_LOGIN_API';
UPDATE dbo.API_MASTER SET Api_Url = N'<path of the employee-by-code API>' WHERE Task_Name =
N'HRP_GET_EMP_DETAIL_CODE';
-- HRMS token, if HRMS needs one (My_Encode of it), per environment:
INSERT INTO dbo.API_AUTHORIZATION (Task_Name, Environment, Api_Authorization) VALUES
(N'HRP_GET_EMP_LOGIN_API', N'Production', N'<My_Encode(token)>');
```

A full URL works too (`Base_Url_Key = NULL`, `Api_Url = https://…`). POGovApi re-reads the rows every 5 minutes. Until they are active, employees see "Sign-in is temporarily unavailable (HRMS did not answer)" and the admin's **Fetch from HRMS** says "HRMS did not answer".

3.2 Settings (`POGovApi/appsettings.json`, section `PoGovSettings`)

Setting	Default	Meaning
`LoginMode`	`Hrms`	`Hrms` = Employee ID + HRMS password. `Local` = email + password kept by the tool (machines without HRMS access)
`AllowAdminLocalLogin`	`true`	The built-in admin (`AdminEmail`) may still sign in with email + local password, for the first set-up and HRMS outages
`AdminEmpCode`	empty	Employee ID of the built-in admin: registered as an HRMS login at start-up
`HrmsPasswordUrl`	MyVishal	"Forgot or locked?" link on the sign-in page
`LoginFailureMinMs`	`1000`	Minimum duration of a failed sign-in
`MaxFailedLogins` / `LockoutMinutes`	5 / 15	Local lockout (checked before HRMS)

3.3 Go-live checklist

1. Run `04_api_master.sql` on the database. Set the `HRMS_API` URL, paths and token (3.1), then restart POGovApi.
2. Sign in as the built-in admin (email). Go to Users → **+ Add user** → an Employee ID → **Fetch from HRMS**. The card must show the employee: this proves POGovApi reaches HRMS.
3. Register the real users (Add user / Bulk add `Employee ID, role`). Link the existing logins with ⚠ **Employee ID needed**.
4. Set `AdminEmpCode` to the admin's Employee ID and restart. Sign in with Employee ID + HRMS password, then set `AllowAdminLocalLogin = false`.
5. Check the Audit log: `login hrms emp=…` for successful sign-ins, `login-fail … not a registered employee` for refused ones.

4. Tests

- `tests/POGovernance.Tests/Platform/HrmsLoginTests.cs` (13):
  - an unregistered employee with the right HRMS password is refused, HRMS is not called and nothing is created;
  - registered employee sign-in;
  - email refused;
  - local lockout before HRMS;
  - HRMS lock (104), outage, answer for another code, resigned employee;
  - inactive login never reaches HRMS;
  - registration checks (unknown / resigned / duplicate / no email / HRMS down);
  - adding by email refused;
  - SCR logins need an ID, then linking;
  - no reset / change of HRMS passwords;
  - built-in admin switch, `AdminEmpCode`, minimum failure time.
- `tests/POGovernance.Tests/Api/HrmsApiClientTests.cs` (4): QPS request body, decoded token, response mapping, row cache, not configured.
- `tests/POGovernance.Tests/SqlServer/ApiMasterSqlServerTests.cs`: HRMS rows are not in `GetAllApi` / `GetSpecificApi`, `SERVER_TASK` resolves the URL, the `*` token is not applied, a task token is.
- End-to-end with the real API and portal against a stand-in HRMS:
  - an unregistered employee with the correct HRMS password was refused and **0 HRMS calls** were made;
  - email sign-in was refused;
  - the admin fetched and added an employee, and fetching the same ID again showed "Already registered";
  - a wrong password was refused, and the right password signed the employee in;
  - the change-password page points to MyVishal;
  - the password never appeared in a log.